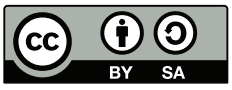
Instruction set of the ULM (Ulm Lecture Machine)



October 11, 2026

Contents

1 Description of the ULM **5**
- 1.1 Data Types . . . 5
- 1.2 Expressing the Interpretation of a Bit Pattern . . . 5
- 1.3 Registers and Virtual Memory . . . 5

2 Directives **7**
- 2.1 .align <expr> . . . 7
- 2.2 .bss . . . 7
- 2.3 .byte <expr> . . . 7
- 2.4 .data . . . 7
- 2.5 .equ <ident>, <expr> . . . 7
- 2.6 .global <ident> . . . 7
- 2.7 .globl <ident> . . . 7
- 2.8 .long <expr> . . . 7
- 2.9 .space <expr> . . . 8
- 2.10 .string <string-literal> . . . 8
- 2.11 .text . . . 8
- 2.12 .word <expr> . . . 8
- 2.13 .quad <expr> . . . 8

3 Instructions **9**
- 3.1 addq . . . 10
- 3.2 andq . . . 12
- 3.3 call . . . 13
- 3.4 getc . . . 14
- 3.5 halt . . . 15
- 3.6 idivq . . . 16
- 3.7 imulq . . . 17
- 3.8 iquotq . . . 18
- 3.9 iremq . . . 19
- 3.10 ja . . . 20
- 3.11 jae . . . 21
- 3.12 jb . . . 22
- 3.13 jbe . . . 23
- 3.14 jge . . . 24
- 3.15 jmp . . . 25
- 3.16 jng . . . 26
- 3.17 jnge . . . 27
- 3.18 jnz . . . 28
- 3.19 jz . . . 29
- 3.20 loads . . . 30
- 3.21 loadz . . . 31

3.22 movb 32
3.23 movq 33
3.24 movsbq 34
3.25 movzbq 35
3.26 mulw 36
3.27 putc 37
3.28 shlq 38
3.29 shrq 39
3.30 subq 40

4 ISA Source File for the ULM Generator 41

Chapter 1

Description of the ULM

1.1 Data Types

Binary digits are called *bits* and have the value 0 or 1. A *bit pattern* is a sequence of bits. For example

$$X := x_{n-1} \dots x_0 \text{ with } x_k \in \{0, 1\} \text{ for } 0 \leq k < n$$

denotes a bit pattern X with n bits. The number of bits in bit pattern is also called its size or width. The ULM architecture defines a *byte* as a bit pattern with 8 bits. Table 1.1 lists ULM's definitions for *word*, *long word*, *quad word* that refer to specific sizes of bit patterns.

1.2 Expressing the Interpretation of a Bit Pattern

For a bit pattern $X = x_{n-1} \dots x_0$ its *unsigned integer* value is expressed and defined through

$$u(X) = u(x_{n-1} \dots x_0) := \sum_{k=0}^{n-1} x_k \cdot 2^k$$

Signed integer values are represented using the *two's complement* and in this respect the notation

$$s(X) = s(x_{n-1}x_{n-2} \dots x_0) := \begin{cases} u(x_{n-2} \dots x_0), & \text{if } x_{n-1} = 0, \\ u(x_{n-2} \dots x_0) - 2^{n-1}, & \text{else} \end{cases}$$

is used.

1.3 Registers and Virtual Memory

The ULM has 256 registers denoted as %0x00, ..., %0xFF. Each of these registers has a width of 64 bits. The %0x00 is a special purpose register and also denoted as *zero register*. Reading form the zero register always gives a bit pattern where all bits have value 0 (zero bit pattern). Writing to the zero register has no effect.

The (virtual) memory of the ULM is an array of 2^{64} memory cells. Each memory cell can store exactly one byte. Each memory cell has an index which is called its *address*. The address is in the range from 0 to 2^{64-1} and the first memory cell of the array has address 0. In notations $M_1(a)$ denotes the memory cell with address a.

Data Size	Size in Bytes	Size in Number of Bits
Bytes	-	8
Word	2	16
Long Word	4	32
Quad Word	8	64

Table 1.1: Names for specific sizes of bit patterns.

1.3.1 Endianness

For referring to data in memory in quantities of words, long words and quad words the definitions

$$\begin{aligned} M_2(a) &:= M_1(a)M_1(a+1) \\ M_4(a) &:= M_2(a)M_2(a+2) \\ M_8(a) &:= M_4(a)M_4(a+4) \end{aligned}$$

are used. The ULM architecture is a *big endian* machine. Therefore we have the equalities

$$\begin{aligned} u(M_2(a)) &= u(M_1(a)M_1(a+1)) \\ u(M_4(a)) &= u(M_2(a)M_2(a+2)) \\ u(M_8(a)) &= u(M_4(a)M_4(a+4)) \end{aligned}$$

1.3.2 Alignment of Data

A quantity of k bytes are aligned in memory if they are stored at an address which is a multiple of k, i. e.

$$M_k(a) \text{ is aligned } \Leftrightarrow a \bmod k = 0$$

Chapter 2

Directives

2.1 .align <expr>

Pad the location counter (in the current segment) to a multiple of <expr>.

2.2 .bss

Set current segment to the BSS segment.

2.3 .byte <expr>

Expression is assembled into next byte.

2.4 .data

Set current segment to the data segment.

2.5 .equ <ident>, <expr>

Updates the symbol table. Sets the value of <ident> to <expr>.

2.6 .global <ident>

Updates the symbol table. Makes the symbol <ident> visible to the linker.

2.7 .globl <ident>

Equivalent to *.globl <ident>*:
Updates the symbol table. Makes the symbol <ident> visible to the linker.

2.8 .long <expr>

Expression <expr> is assembled into next long word (4 bytes).

2.9 .space <expr>

Emits <expr> bytes. Each byte with value 0x00.

2.10 .string <string-literal>

Emits bytes for the zero-terminated <string-literal>.

2.11 .text

Set current segment to the text segment.

2.12 .word <expr>

Expression <expr> is assembled into next word (2 bytes).

2.13 .quad <expr>

Expression <expr> is assembled into next quad word (8 bytes).

Chapter 3

Instructions

3.1 addq

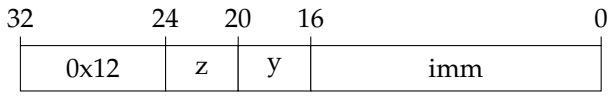
3.1.1 Assembly Notation

addq imm, %y, %z

Purpose

Adds the zero extended immediate value `imm` to register `%y` and stores the result in register `%z`.

Format



Effect

$$(u(\%y) + u(imm)) \bmod 2^{64} \rightarrow u\,(\%z)$$

Updates the status flags:

Flag	Condition
ZF	$u(\%y) + u(imm) \bmod 2^{64} = 0$
CF	$u(\%y) + u(imm) \geq 2^{64}$
OF	$s(\%y) + s(imm) \notin \{-2^{63}, \ldots, 2^{63}\}$
SF	$s(\%y) + s(imm) < 0$

3.1.2 Assembly Notation

addq %x, %y, %z

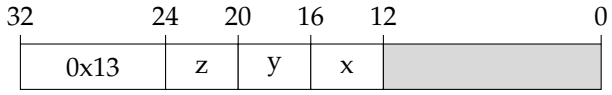
Alternative Assembly Notation

movq %x, %z

Purpose

Adds register `%x` to register and stores the result in register `%z`.

Format



Effect

$$(u(\%y) + u(\%x)) \bmod 2^{64} \rightarrow u\,(\%z)$$

Updates the status flags:

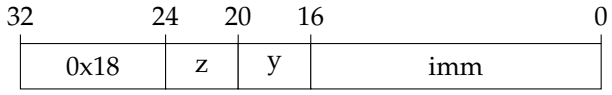
Flag	Condition
ZF	$u(\%y) + u(\%x) \bmod 2^{64} = 0$
CF	$u(\%y) + u(\%x) \geq 2^{64}$
OF	$s(\%y) + s(\%x) \notin \{-2^{63}, \ldots, 2^{63}\}$
SF	$s(\%y) + s(\%x) < 0$

3.2 andq

3.2.1 Assembly Notation

andq imm, %y, %z

Format



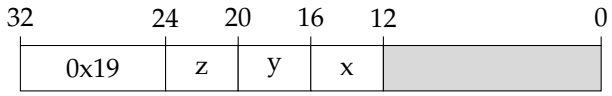
Effect

$u(imm) \wedge_b u(\%y) \rightarrow u\,(\%z)$

3.2.2 Assembly Notation

andq %x, %y, %z

Format



Effect

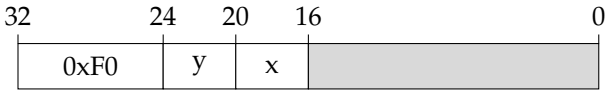
$u(\%x) \wedge_b u(\%y) \rightarrow u\,(\%z)$

3.3 call

3.3.1 Assembly Notation

call %x

Format



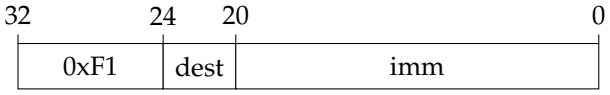
Effect

$$(u(\%IP) + 4) \bmod 2^{64} \rightarrow u(\%x)$$
$$u(\%x) \rightarrow u(\%IP)$$

3.3.2 Assembly Notation

call imm, %dest

Format



Effect

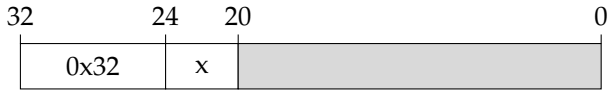
$$(u(\%IP) + 4) \bmod 2^{64} \rightarrow u(\%dest)$$
$$u(imm) \rightarrow u(\%IP)$$

3.4 getc

3.4.1 Assembly Notation

getc %x

Format



Effect

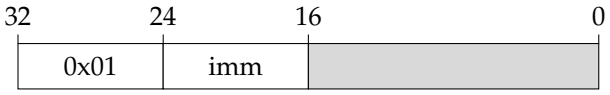
$s(ulm_readChar()) \wedge_b 255 \bmod 2^{64} \rightarrow u\ (\%x)$

3.5 halt

3.5.1 Assembly Notation

halt imm

Format



Effect

halt program execution with exit code $u(imm) \bmod 2^8$

3.5.2 Assembly Notation

halt %x

Format



Effect

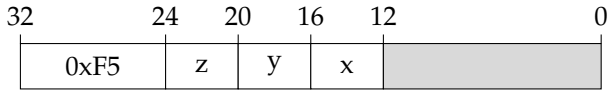
halt program execution with exit code $u(\%x) \bmod 2^8$

3.6 idivq

3.6.1 Assembly Notation

idivq %x, %y, %z

Format



Effect

Computes the divisor

$$\left\lfloor \frac{u(\%y)}{u(\%x)} \right\rfloor_0 \rightarrow u\,(\%z)$$

and the remainder

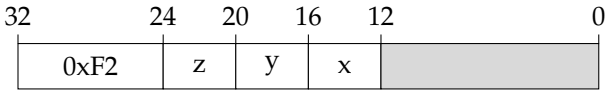
$$u(\%y) \bmod u(\%x) \rightarrow u\,(\%\{u(z)+1\})$$

3.7 imulq

3.7.1 Assembly Notation

imulq %x, %y, %z

Format



Effect

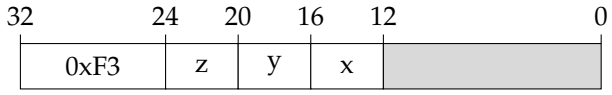
$$u(\%x) \cdot u(\%y) \bmod 2^{64} \rightarrow u\,(\%z)$$

3.8 iquotq

3.8.1 Assembly Notation

iquotq %x, %y, %z

Format



Effect

Computes the divisor

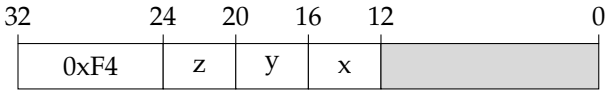
$$\left\lfloor \frac{u(\%y)}{u(\%x)} \right\rfloor_0 \rightarrow u\,(\%z)$$

3.9 iremq

3.9.1 Assembly Notation

iremq %x, %y, %z

Format



Effect

Computes the remainder

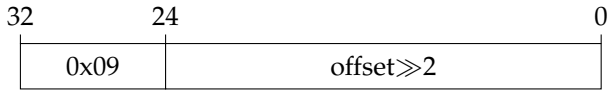
$$u(\%y) \bmod u(\%x) \rightarrow u\,(\%z)$$

3.10 ja

3.10.1 Assembly Notation

ja offset

Format



Effect

If the condition

$$ZF = 0 \wedge CF = 0$$

evaluates to true then

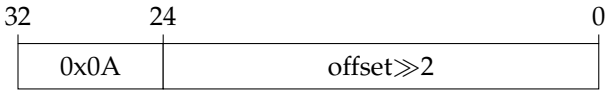
$$(u(\%IP) + s(offset)) \bmod 2^{64} \rightarrow u(\%IP)$$

3.11 jae

3.11.1 Assembly Notation

jae offset

Format



Effect

If the condition

$$CF = 0$$

evaluates to true then

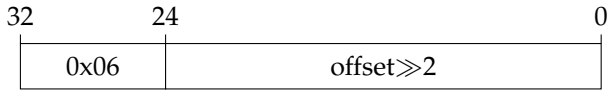
$$(u(\%IP) + s(offset)) \bmod 2^{64} \rightarrow u(\%IP)$$

3.12 jb

3.12.1 Assembly Notation

jb offset

Format



Effect

If the condition

$$CF = 1$$

evaluates to true then

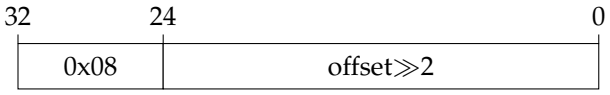
$$(u(\%IP) + s(offset)) \bmod 2^{64} \rightarrow u(\%IP)$$

3.13 jbe

3.13.1 Assembly Notation

jbe offset

Format



Effect

If the condition

$$ZF = 1 \vee CF = 1$$

evaluates to true then

$$(u(\%IP) + s(offset)) \bmod 2^{64} \rightarrow u(\%IP)$$

3.14 jge

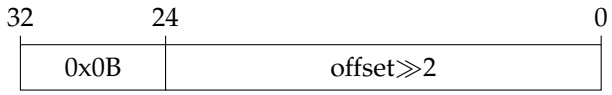
3.14.1 Assembly Notation

jge offset

Alternative Assembly Notation

jnl offset

Format



Effect

If the condition

$$SF = OF$$

evaluates to true then

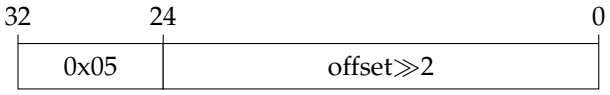
$$(u(\%IP) + s(offset)) \bmod 2^{64} \rightarrow u(\%IP)$$

3.15 jmp

3.15.1 Assembly Notation

jmp offset

Format



Effect

$$(u(\%IP) + s(offset)) \bmod 2^{64} \rightarrow u(\%IP)$$

3.15.2 Assembly Notation

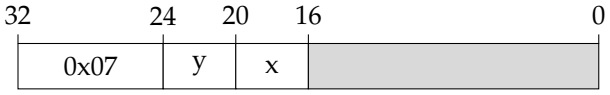
jmp %y, %x

Alternative Assembly Notation

call %y, %x

ret %y

Format



Effect

$$\begin{aligned} (u(\%IP) + 4) \bmod 2^{64} &\rightarrow u(\%x) \\ u(\%y) &\rightarrow u(\%IP) \end{aligned}$$

3.16 jng

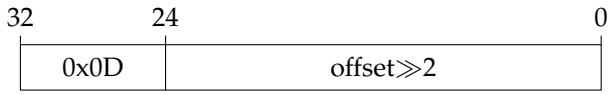
3.16.1 Assembly Notation

jng offset

Alternative Assembly Notation

jle offset

Format



Effect

If the condition

$$ZF = 1 \vee SF \neq OF$$

evaluates to true then

$$(u(\%IP) + s(offset)) \bmod 2^{64} \rightarrow u(\%IP)$$

3.17 jnge

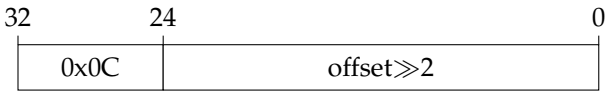
3.17.1 Assembly Notation

jnge offset

Alternative Assembly Notation

jl offset

Format



Effect

If the condition

$$SF \neq OF$$

evaluates to true then

$$(u(\%IP) + s(offset)) \bmod 2^{64} \rightarrow u(\%IP)$$

3.18 jnz

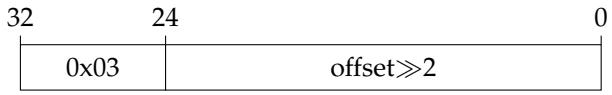
3.18.1 Assembly Notation

jnz offset

Alternative Assembly Notation

jne offset

Format



Effect

If the condition

$$ZF = 0$$

evaluates to true then

$$(u(\%IP) + s(offset)) \bmod 2^{64} \rightarrow u(\%IP)$$

3.19 jz

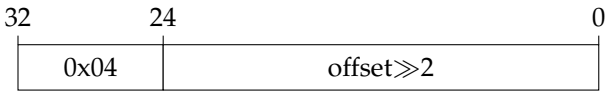
3.19.1 Assembly Notation

jz offset

Alternative Assembly Notation

je offset

Format



Effect

If the condition

$$ZF = 1$$

evaluates to true then

$$(u(\%IP) + s(offset)) \bmod 2^{64} \rightarrow u(\%IP)$$

3.20 loads

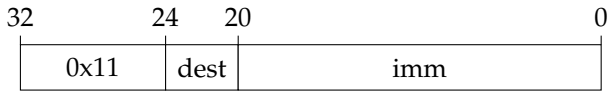
3.20.1 Assembly Notation

loads imm, %dest

Purpose

Load sign extended immediate value into destination register.

Format



Effect

$s(imm) \bmod 2^{64} \rightarrow u\,(\%dest)$

3.21 loadz

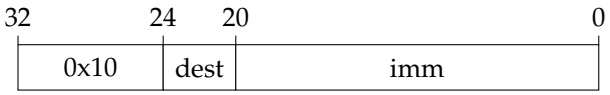
3.21.1 Assembly Notation

loadz imm, %dest

Purpose

Load zero extended immediate value into destination register.

Format



Effect

$$u(imm) \bmod 2^{64} \rightarrow u\,(\%dest)$$

3.22 movb

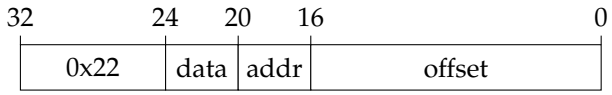
3.22.1 Assembly Notation

movb %data, offset(%addr)

Alternative Assembly Notation

movb %data, (%addr)

Format



Effect

$$u(\%data) \bmod 2^8 \rightarrow u\left(\mathrm{M}_1\left(addr\right)\right) \text{ with } addr = \left(s(offset) + u(\%addr)\right) \bmod 2^{64}$$

3.23 movq

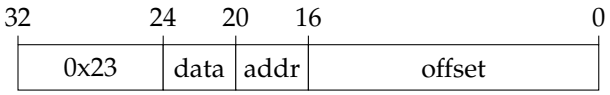
3.23.1 Assembly Notation

movq offset(%addr), %data

Alternative Assembly Notation

movq (%addr), %data

Format



Effect

$$u\left(\mathrm{M}_8\left(addr\right)\right) \rightarrow u\left(\%data\right) \text{ with } addr = \left(s(offset) + u(\%addr)\right) \bmod 2^{64}$$

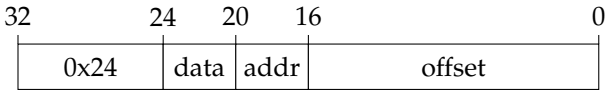
3.23.2 Assembly Notation

movq %data, offset(%addr)

Alternative Assembly Notation

movq %data, (%addr)

Format



Effect

$$u(\%data) \bmod 2^{64} \rightarrow u\left(\mathrm{M}_8\left(addr\right)\right) \text{ with } addr = \left(s(offset) + u(\%addr)\right) \bmod 2^{64}$$

3.24 movsbq

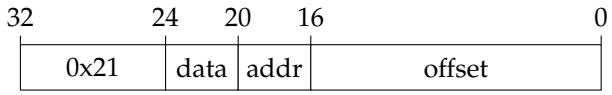
3.24.1 Assembly Notation

movsbq offset(%addr), %data

Alternative Assembly Notation

movsbq (%addr), %data

Format



Effect

$$s\left(\mathrm{M}_1\left(addr\right)\right) \rightarrow u\left(\%data\right) \text{ with } addr = \left(s(offset) + u(\%addr)\right) \bmod 2^{64}$$

3.25 movzbq

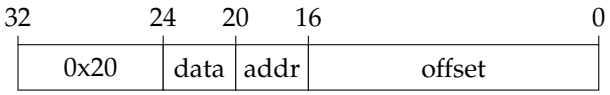
3.25.1 Assembly Notation

movzbq offset(%addr), %data

Alternative Assembly Notation

movzbq (%addr), %data

Format



Effect

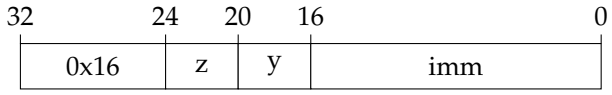
$$u\left(\mathrm{M}_1\left(addr\right)\right) \rightarrow u\left(\%data\right) \text{ with } addr = \left(s(offset) + u(\%addr)\right) \bmod 2^{64}$$

3.26 mulw

3.26.1 Assembly Notation

mulw imm, %y, %z

Format



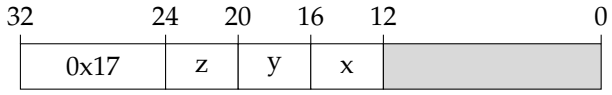
Effect

$$u(imm) \cdot u(\%y) \wedge_b 65535 \bmod 2^{64} \rightarrow u\,(\%z)$$

3.26.2 Assembly Notation

mulw %x, %y, %z

Format



Effect

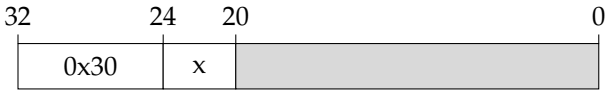
$$u(\%x) \wedge_b 65535 \cdot u(\%y) \wedge_b 65535 \bmod 2^{64} \rightarrow u\,(\%z)$$

3.27 putc

3.27.1 Assembly Notation

putc %x

Format



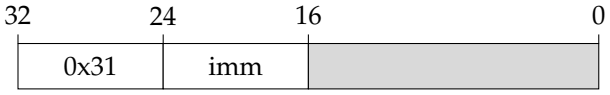
Effect

ulm_printChar(%*x*)

3.27.2 Assembly Notation

putc imm

Format



Effect

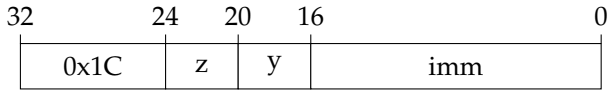
ulm_printChar(*imm*)

3.28 shlq

3.28.1 Assembly Notation

shlq imm, %y, %z

Format



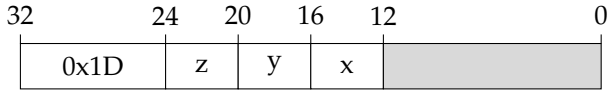
Effect

TODO: ulm_shiftLeft64

3.28.2 Assembly Notation

shlq %x, %y, %z

Format



Effect

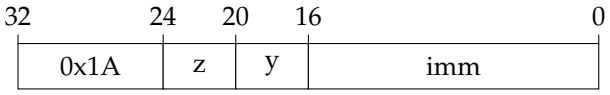
TODO: ulm_shiftLeft64

3.29 shrq

3.29.1 Assembly Notation

shrq imm, %y, %z

Format



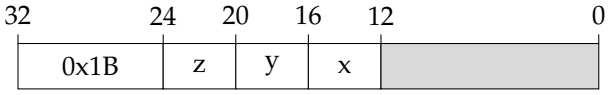
Effect

TODO: ulm_shiftRightUnsigned64

3.29.2 Assembly Notation

shrq %x, %y, %z

Format



Effect

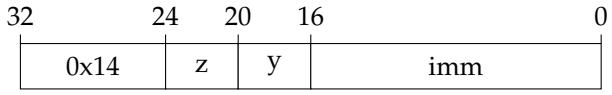
TODO: ulm_shiftRightUnsigned64

3.30 subq

3.30.1 Assembly Notation

subq imm, %y, %z

Format



Effect

$$(u(\%y) - u(imm)) \bmod 2^{64} \rightarrow u\,(\%z)$$

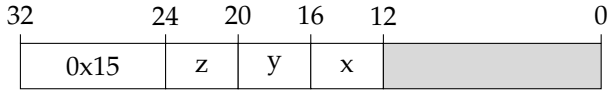
Updates the status flags:

Flag	Condition
ZF	$u(\%y) - u(imm) = 0$
CF	$u(\%y) - u(imm) < 0$
OF	$s(\%y) - s(imm) \notin \{-2^{63}, \ldots, 2^{63}\}$
SF	$s(\%y) - s(imm) < 0$

3.30.2 Assembly Notation

subq %x, %y, %z

Format



Effect

$$(u(\%y) - u(\%x)) \bmod 2^{64} \rightarrow u\,(\%z)$$

Updates the status flags:

Flag	Condition
ZF	$u(\%y) - u(\%x) = 0$
CF	$u(\%y) - u(\%x) < 0$
OF	$s(\%y) - s(\%x) \notin \{-2^{63}, \ldots, 2^{63}\}$
SF	$s(\%y) - s(\%x) < 0$

Chapter 4

ISA Source File for the ULM Generator

```
U8 (OP u 8) (imm u 8)
R (OP u 8) (x u 4)
RR (OP u 8) (y u 4) (x u 4)
REL_JMP (OP u 8) (offset j 24)
U20_R (OP u 8) (dest u 4) (imm u 20)
S20_R (OP u 8) (dest u 4) (imm s 20)
R_R_R (OP u 8) (z u 4) (y u 4) (x u 4)
U16_R_R (OP u 8) (z u 4) (y u 4) (imm u 16)
S16_R_R (OP u 8) (z u 4) (y u 4) (imm s 16)
MR_R (OP u 8) (data u 4) (addr u 4) (offset s 16)
R_MR (OP u 8) (data u 4) (addr u 4) (offset s 16)

#
# CU (control unit) instructions
#

0x01 U8
: halt imm
        ulm_halt(imm);

0x02 R
: halt %x
        ulm_halt(ulm_regVal(x));

0x03 REL_JMP
: jnz offset
: jne offset
        ulm_conditionalRelJump(ulm_statusReg[ULM_ZF] == 0, offset);

0x04 REL_JMP
: jz offset
: je offset
        ulm_conditionalRelJump(ulm_statusReg[ULM_ZF] == 1, offset);

0x05 REL_JMP
```

```
36 : jmp offset
37       ulm_unconditionalRelJump(offset);
38
39 0x06 REL_JMP
40 : jb offset
41       ulm_conditionalRelJump(ulm_statusReg[ULM_CF] == 1, offset);
42
43 0x07 RR
44 : jmp %y, %x
45 : call %y, %x
46 : ret %y
47       ulm_absJump(ulm_regVal(y), x);
48
49 0x08 REL_JMP
50 : jbe offset
51       ulm_conditionalRelJump(ulm_statusReg[ULM_ZF] == 1
52                              || ulm_statusReg[ULM_CF] == 1, offset);
53
54 0x09 REL_JMP
55 : ja offset
56       ulm_conditionalRelJump(ulm_statusReg[ULM_ZF] == 0
57                              && ulm_statusReg[ULM_CF] == 0, offset);
58
59 0x0A REL_JMP
60 : jae offset
61       ulm_conditionalRelJump(ulm_statusReg[ULM_CF] == 0, offset);
62
63 0x0B REL_JMP
64 : jge offset
65 : jnl offset
66       ulm_conditionalRelJump(ulm_statusReg[ULM_SF] == ulm_statusReg[ULM_OF],
67                              offset);
68
69 0x0C REL_JMP
70 : jnge offset
71 : jl offset
72       ulm_conditionalRelJump(ulm_statusReg[ULM_SF] != ulm_statusReg[ULM_OF],
73                              offset);
74
75 0x0D REL_JMP
76 : jng offset
77 : jle offset
78       ulm_conditionalRelJump(ulm_statusReg[ULM_ZF] == 1 ||
79                              ulm_statusReg[ULM_SF] != ulm_statusReg[ULM_OF],
80                              offset);
81
82
83 #
84 # ALU (arithmetic logic unit)
85 #
86
87 0x10 U20_R
88 # Load zero extended immediate value into destination register.
```

```
: loadz imm, %dest
    ulm_setReg(imm, dest);

0x11 S20_R
# Load sign extended immediate value into destination register.
: loads imm, %dest
    ulm_setReg(imm, dest);

0x12 U16_R_R
# Adds the zero extended immediate value \\texttt{imm} to register
# \\texttt{%y} and stores the result in register \\texttt{%z}.
: addq imm, %y, %z
    ulm_add64(imm, ulm_regVal(y), z);

0x13 R_R_R
# Adds register \\texttt{%x} to register and stores the result in
# register \\texttt{%z}.
: addq %x, %y, %z
: movq %x, %z
    ulm_add64(ulm_regVal(x), ulm_regVal(y), z);

0x14 U16_R_R
: subq imm, %y, %z
    ulm_sub64(imm, ulm_regVal(y), z);

0x15 R_R_R
: subq %x, %y, %z
    ulm_sub64(ulm_regVal(x), ulm_regVal(y), z);

0x16 U16_R_R
: mulw imm, %y, %z
    ulm_mul64(imm, ulm_regVal(y) & 0xFFFF, z);

0x17 R_R_R
: mulw %x, %y, %z
    ulm_mul64(ulm_regVal(x) & 0xFFFF, ulm_regVal(y) & 0xFFFF, z);

0x18 U16_R_R
: andq imm, %y, %z
    ulm_and64(imm, ulm_regVal(y), z);

0x19 R_R_R
: andq %x, %y, %z
    ulm_and64(ulm_regVal(x), ulm_regVal(y), z);

0x1A U16_R_R
: shrq imm, %y, %z
    ulm_shiftRightUnsigned64(imm, ulm_regVal(y), z);

0x1B R_R_R
: shrq %x, %y, %z
    ulm_shiftRightUnsigned64(ulm_regVal(x), ulm_regVal(y), z);

```

```
0x1C U16_R_R
: shlq imm, %y, %z
      ulm_shiftLeft64(imm, ulm_regVal(y), z);

0x1D R_R_R
: shlq %x, %y, %z
      ulm_shiftLeft64(ulm_regVal(x), ulm_regVal(y), z);

#
# bus instructions
#

0x20 MR_R
: movzbq offset(%addr), %data
: movzbq (%addr), %data
      ulm_fetch64(offset, addr, 0, 1, ULM_ZERO_EXT, 1, data);

0x21 MR_R
: movsbq offset(%addr), %data
: movsbq (%addr), %data
      ulm_fetch64(offset, addr, 0, 1, ULM_SIGN_EXT, 1, data);

0x22 R_MR
: movb %data, offset(%addr)
: movb %data, (%addr)
      ulm_store64(offset, addr, 0, 0, 1, data);

0x23 MR_R
: movq offset(%addr), %data
: movq (%addr), %data
      ulm_fetch64(offset, addr, 0, 1, ULM_ZERO_EXT, 8, data);

0x24 R_MR
: movq %data, offset(%addr)
: movq %data, (%addr)
      ulm_store64(offset, addr, 0, 0, 8, data);

#
# i/o instructions
#
0x30 R
: putc %x
      ulm_printChar(ulm_regVal(x));

0x31 U8
: putc imm
      ulm_printChar(imm);

0x32 R
: getc %x
      ulm_setReg(ulm_readChar() & 0xFF, x);

# HPC0 extensions
```

```
195
196 0xF0 RR
197 : call %x
198         ulm_absJump(ulm_regVal(x), x);
199
200 0xF1 U20_R
201 : call imm, %dest
202         ulm_absJump(imm, dest);
203
204 0xF2 R_R_R
205 : imulq %x, %y, %z
206         ulm_mul64(ulm_regVal(x), ulm_regVal(y), z);
207
208 0xF3 R_R_R
209 : iquotq %x, %y, %z
210         ulm_iquot64(ulm_regVal(x), ulm_regVal(y), z);
211
212 0xF4 R_R_R
213 : iremq %x, %y, %z
214         ulm_irem64(ulm_regVal(x), ulm_regVal(y), z);
215
216 0xF5 R_R_R
217 : idivq %x, %y, %z
218         ulm_idiv64(ulm_regVal(x), ulm_regVal(y), z);
```